

Trong các dự án EdTech (Đặc biệt là hệ thống Adaptive Learning - Học tập thích ứng), sự thất bại hiếm khi nằm ở code thuật toán, mà thường nằm ở thảm họa **GIGO (Garbage In, Garbage Out - Đầu vào rác thì Đầu ra rác)**.

Sẽ có một sự "lệch pha" rất lớn về ngôn ngữ:

- **Tech Team** nói bằng Toán học: *Tham số IRT $\$b$, Xác suất lụi $\$c$, BKT $\$P(T)$, Đồ thị DAG.*
- **Academic Team** nói bằng Sư phạm: *Mức độ nhận thức (Nhận biết/Vận dụng), Format câu hỏi, Bám sát SGK, Mất gốc.*

Nếu bạn đưa thẳng Database Schema hoặc file JSON bắt đội Academic điền các con số thập phân, dự án sẽ sụp đổ. Nhiệm vụ của bạn là xây dựng một hệ thống **Internal CMS (Content Management System)** đóng vai trò là "**Máy phiên dịch**" và một **Môi trường Testing** đặc thù.

Dưới đây là bản thiết kế chi tiết các Requirements và System Design cho bài toán này:

PHẦN 1: CÁC REQUIREMENTS CỐT LÕI (BACKEND GUARDRAILS)

Để bảo vệ "Não bộ AI" khỏi những sai sót nhập liệu của con người, hệ thống cần các rào cản (Guardrails) sau:

1. Lớp Phiên Dịch (Auto-Translation / Mapping Layer)

Bạn tuyệt đối không cho phép Academic nhập các biến IRT/BKT. Thay vào đó, tạo Dropdown ngôn ngữ tự nhiên, Backend sẽ tự map ra số cho giai đoạn Cold-start (chưa có data):

- **Với Độ khó (IRT $\$b$):**
 - Họ chọn *Dễ* $\rightarrow$ Code tự gán $\$b = -1.0$
 - Họ chọn *Trung bình* $\rightarrow$ Code tự gán $\$b = 0.0$
 - Họ chọn *Khó / Vận dụng cao* $\rightarrow$ Code tự gán $\$b = 1.5$
- **Với Xác suất lụi (IRT $\$c$):**
 - Họ chọn *Trắc nghiệm 4 đáp án* $\rightarrow c = 0.25$
 - Họ chọn *Điền từ / Tự luận* $\rightarrow c = 0.0$ (Không thể đoán mò)
- **Với Độ hiệu quả của bài giảng (BKT $\$P(T)$):**
 - Họ gán Tag *Video giải chi tiết* $\rightarrow P(T) = 0.8$ (Xem xong tỉ lệ hiểu rất cao).
 - Họ gán Tag *Text tóm tắt* $\rightarrow P(T) = 0.4$ (Tỉ lệ hiểu thấp hơn, thanh Tiến độ sẽ tăng chậm hơn).

2. Tính Bất biến của Dữ liệu (Data Immutability & Soft Delete)

- **Vấn đề:** Đang có 1000 học sinh chạy trên hệ thống. Giáo viên phát hiện 1 câu hỏi bị sai đề, liền vào CMS sửa lại đáp án và đổi độ khó. Lập tức, lịch sử điểm θ của hàng ngàn học sinh cũ bị sai lệch thuật toán.
- **Requirement sống còn:** Không bao giờ cấp quyền **UPDATE** nội dung đề bài/đáp án/độ khó cho bảng **Items**. Nếu sai, chỉ cho phép **Soft Delete** (**is_active = false**), hệ thống sẽ tạo ra 1 phiên bản Câu hỏi mới (Item Version 2) để máy học lại từ đầu. Học sinh mới làm câu mới, điểm học sinh cũ giữ nguyên.

3. Bộ lọc tính toán vẹn của Đồ thị KST (Graph Integrity)

- **Chống chu trình (Anti-Cycle):** Tại API **POST /prerequisite**, trước khi lưu Database, bắt buộc chạy hàm **nx.is_directed_acyclic_graph()**. Nếu phát hiện vòng lặp (A đòi B, B đòi C, C lại đòi A) $\rightarrow$ Block API và báo lỗi ngay.

- **Chống ngõ cụt (Dead-end check):** Cảnh báo nếu có một Node/KC nào đó có dưới 10 câu hỏi, hoặc toàn câu hỏi mức độ "Khó". Vì nếu học sinh yếu rớt vào Node này, hệ thống sẽ hết câu hỏi Dễ để gấp ra cho học sinh làm $\rightarrow$ Hệ thống bị Crash/Kẹt.

PHẦN 2: THIẾT KẾ INTERFACE NHẬP LIỆU (AUTHORING UI)

Bạn cần xây dựng một CMS nội bộ với 2 màn hình chính:

1. Visual Graph Builder (Xây dựng Cây kiến thức)

Tuyệt đối **không dùng dạng Bảng (Table)** để bắt giáo viên nhập ID bài học Tiên quyết. Mất người không thể dò bảng để tưởng tượng ra cấu trúc cây.

- **Tech Stack:** Dùng các thư viện UI Node-based như **React Flow**.
- **Trải nghiệm:** Giáo viên dùng chuột nắm kéo các Box (đại diện cho bài học) và kéo mũi tên nối chúng lại (Giống phần mềm Miro / Mindmap). Nối sai logic tạo vòng lặp $\rightarrow$ Dây đứt và báo đỏ ngay trên UI.

2. Item Bank Editor (Ngân hàng Câu hỏi)

- Tích hợp Rich-text Editor hỗ trợ **LaTeX/KaTeX** để gõ công thức Toán/Lý/Hóa.
- Giao diện nhập theo form: Chọn Node Kiến thức $\rightarrow$ Điền đề $\rightarrow$ Chọn mức độ (Dễ/TB/Khó) để kích hoạt *Mapping Layer* ở phần 1.

PHẦN 3: CÁCH CHẠY TESTING (GIỮA TECH VÀ ACADEMIC)

Với một hệ thống Adaptive linh hoạt (Mỗi học sinh ra 1 luồng khác nhau), kiểu test QA truyền thống "*Click Next xem có ra đúng câu hỏi bài 2 không*" là vô dụng. Bạn phải test qua 2 tầng:

Tầng 1: Chạy Bot Tự Động (Auto Simulation)

Bạn (Tech Lead) viết 1 script Python tạo ra 3 loại "Học sinh ảo" (Personas Bot) bắn API liên tục vào Engine:

1. **Bot Thần Đồng:** Script luôn submit **correct = true**.
 - *Kỳ vọng (Assert):* Thuật toán chỉ móc đúng những câu Khó nhất ra test, kết thúc vòng lặp cực nhanh (VD: 10 câu), Rank θ vọt lên đỉnh, pass toàn bộ Graph.
2. **Bot Mất Gốc:** Luôn submit **correct = false**.
 - *Kỳ vọng:* Hệ thống dò giật lùi đồ thị nhanh chóng (Navigate down) và rớt xuống tận Root Node nền tảng dưới đáy, báo cáo "Lỗi hỏng kiến thức gốc".
3. **Bot Nhắm Mắt Bầm Bờ:** Random đáp án 25% True / 75% False.
 - *Kỳ vọng:* Toán học Bayes nhận diện được đang đoán lụi, thanh BKT $P(L)$ lết lên cực kỳ chậm, hệ thống khóa chặt Bot này không cho qua bài.

(Trích xuất log của Bot này ra file Excel đưa cho Academic duyệt luồng chạy).

Tầng 2: Môi trường UAT "God-Mode" (Dành cho Academic test tay)

Academic sẽ không tin máy móc nếu họ không tự mình trải nghiệm. Hãy code một màn hình **X-Ray Sandbox (Nhìn xuyên thấu AI)** chia làm 2 nửa trên web:

- **Nửa bên trái (Giao diện Học sinh):** Hiện thị câu hỏi/video bài giảng như thật. Có 2 nút bọc ngoài: **[Cố tình làm ĐÚNG]** và **[Cố tình làm SAI]** để giáo viên đi bài nhanh không cần giải toán.
- **Nửa bên phải (Não bộ AI - Console):** Màn hình hiển thị realtime suy nghĩ của máy tính:
 - *Rank (Theta) hiện tại:* Đang từ 0.0 tụt xuống -0.5.
 - *Tiến độ (P_L) bài này:* $45\% \rightarrow$ Cần 95% để pass.

- *AI Explainability Log*: "Vừa bốc câu ID #88 vì học sinh vừa làm sai, cần bốc câu Dễ ($b=-1.0$) để hồi phục lại tỉ lệ thắng 65%".

Khi giáo viên tự tay bấm và thấy hệ thống hành xử, tự lùi bài, tự bốc câu khó/dễ đúng y như logic sư phạm của họ, họ sẽ hoàn toàn tin tưởng vào hệ thống của bạn.

Tầng 3: Hiệu chuẩn Data sau khi Launch (Calibration Feedback Loop)

Sau 1 tháng chạy có data học sinh thật, bạn build một trang **Anomaly Report (Báo cáo bất thường)**:

- *Hệ thống báo*: "Câu hỏi ID #150 team Academic gán là **ĐỀ**, nhưng data thực tế 80% học sinh **LÀM SAI**. Đề xuất tự động cập nhật độ khó thành **RẤT KHÓ**. Chấp nhận (Y/N)?"
- Đây chính là sức mạnh tự học của AI, hệ thống sẽ ngày càng chính xác hơn những gì con người nhập ban đầu.